

UE : POO - Programmation Orientée Objet JAVA

Outil de gestion d'images

Étudiantes :

Lylia IMLOUL
Lyna HADJI

Encadrant :

Emmanuel CONCHON

1. Introduction....3
2. Lancement de l'application....4
2.1.Présentation....4
2.2.Chargement de l'interface....4
2.3.Création de la scène....4
2.4.Application du style CSS....4
2.5.Affichage de la fenêtre....4
2.6.Code associé....4
3. Architecture du projet....5
3.1.Modèle MVC....5
3.2.Organisation du projet....5
3.3.Flux de fonctionnement....6
4. Fonctionnalités....6
4.1.Chargement d'image....7
4.2.Application de filtres....7
4.3.Transformation d'image....7
4.4.Gestion des tags....7
4.5.Sauvegarde des données....7
4.6.Sécurité (chiffrement et déchiffrement)....8
4.7.Interface utilisateur....8
5. Détails techniques – Filtres d'image....8
5.1.Filtre Gray (niveaux de gris)....8
5.2.Filtre Prewitt (détection de contours)....9
5.3.Filtre Sepia....9
5.4.Filtre Swap....9
5.5.Transformations d'image....9
5.6.Sérialisation des données (JSON)....10
5.7.Chiffrement et déchiffrement....10
5.8.Gestion des erreurs....11
6. Limites du projet....11
7. Conclusion....11
8. Bibliographie....12

1. Introduction

Ce projet consiste à développer une application de manipulation d'images en utilisant Java et la bibliothèque JavaFX.

L'objectif est de permettre à l'utilisateur de charger une image, d'appliquer différents filtres, d'effectuer des transformations, d'ajouter des tags et de sauvegarder les informations associées.

Ce projet met en pratique plusieurs notions importantes :

- la programmation orientée objet (POO)
- l'architecture MVC (Model-View-Controller)
- la manipulation d'images
- la sérialisation des données en JSON
- des notions de sécurité (chiffrement)

2. Lancement de l'application

2.1.Présentation

L'application démarre grâce à la classe **MainApp**, qui constitue le point d'entrée du programme.
Cette classe hérite de la classe Application fournie par JavaFX, ce qui permet de créer et gérer une interface graphique.
La méthode start(Stage stage) est automatiquement appelée au lancement de l'application.

2.2.Chargement de l'interface

L'interface graphique est définie dans un fichier FXML (main.fxml).

Ce fichier est chargé à l'aide de la classe FXMLLoader, ce qui permet de séparer la structure de l'interface du code Java. Cette séparation rend le code plus clair et plus maintenable.

2.3.Création de la scène

Une scène (Scene) est ensuite créée à partir du contenu chargé.

La scène représente l'ensemble des éléments graphiques affichés à l'écran.

2.4.Application du style CSS

Un fichier CSS est appliqué à la scène afin de personnaliser l'apparence de l'interface.

Cela permet de modifier les couleurs, les boutons et la mise en page, améliorant ainsi l'expérience utilisateur.

2.5.Affichage de la fenêtre

La scène est associée à la fenêtre principale (Stage).
Le titre de la fenêtre est défini avec la méthode setTitle(). Enfin, la méthode show() permet d'afficher l'application à l'écran.

2.6.Code associé

```
package app;
import javafx.application.Application; // pour creer une app graphic (classe de base de
javaFX)
import javafx.fxml.FXMLLoader; // charger une fichier FXML
import javafx.scene.Scene; // contenu de la fenetre
import javafx.stage.Stage; // reprenste la fenetre
public class MainApp extends Application {
    @Override
    public void start(Stage stage) throws Exception { // methode appelle auto au
demarrage
        FXMLLoader loader = new FXMLLoader(getClass().getResource("/view/main.fxml"));
      Scene scene = new Scene(loader.load());
        scene.getStylesheets().add(
                getClass().getResource("/styles/style.css").toExternalForm()
        );
```

3. Architecture du projet

L'application développée repose sur une architecture logicielle claire et structurée afin de faciliter sa compréhension, sa maintenance et son évolution.
Elle suit le modèle MVC (Model-View-Controller), qui permet de séparer les responsabilités en différentes couches indépendantes.

3.1.Modèle MVC

Le modèle MVC est une architecture largement utilisée dans le développement d'applications graphiques. Il se compose de trois parties principales :

- Model :

Le modèle représente les données et la logique métier de l'application.
Dans ce projet, il correspond aux classes telles que ImageModel et TaggableImage, qui gèrent respectivement l'image, les transformations appliquées, ainsi que les tags associés.

- View :

La vue correspond à l'interface graphique de l'application.
Elle est définie dans un fichier FXML (main.fxml) et contient les éléments visuels tels que les boutons, le champ de texte et la zone d'affichage de l'image (ImageView).

- Controller :

Le contrôleur assure la communication entre la vue et le modèle.
La classe MainController gère les actions de l'utilisateur (clics sur les boutons, saisie de texte) et appelle les méthodes appropriées du modèle pour effectuer les traitements.

Cette séparation permet d'avoir un code plus lisible, mieux organisé et plus facile à modifier.

3.2.Organisation du projet

Le projet est structuré en plusieurs packages afin de regrouper les classes selon leur rôle :

- app : contient la classe principale MainApp qui permet de lancer l'application JavaFX
- controller : contient la classe MainController qui gère les interactions utilisateur
- model : contient les classes liées aux données, comme ImageModel et TaggableImage
- model.filter : contient les filtres d'image (Gray, Sepia, Prewitt, Swap)
- model.transform : contient les transformations (rotation, symétrie)
- security : contient les classes liées au chiffrement, notamment CryptoUtil
- util : contient les classes utilitaires, comme la gestion du JSON (JsonUtil)

Cette organisation améliore la lisibilité du code et permet une meilleure séparation des responsabilités.

3.3.Flux de fonctionnement

Le fonctionnement de l'application repose sur une interaction entre les différentes couches du modèle MVC.

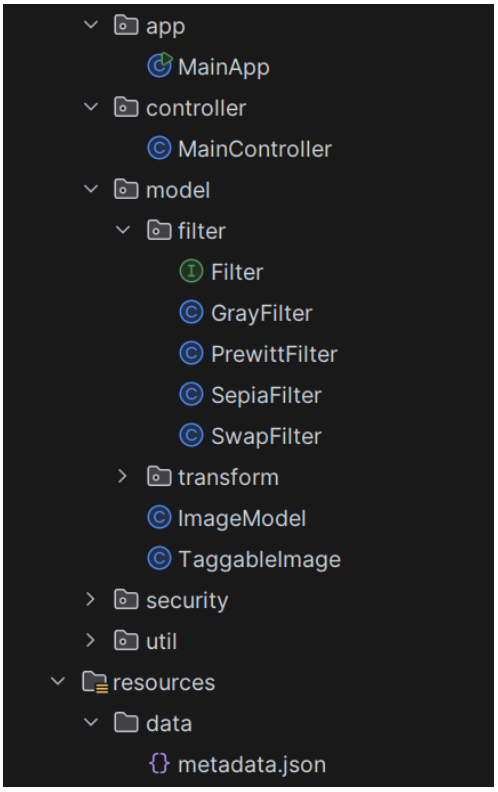
Lorsqu'un utilisateur effectue une action, le processus est le suivant :

1) L'utilisateur interagit avec l'interface graphique (View), par exemple en cliquant sur un bouton
2) Le contrôleur (Controller) reçoit cette action
3) Le contrôleur appelle le modèle (Model) pour effectuer le traitement demandé (filtre, transformation, etc.)
4) Le résultat est ensuite renvoyé et affiché dans l'interface (View)

Ce fonctionnement permet une communication claire et organisée entre les différentes parties de l'application.

L'utilisation du modèle MVC ainsi que l'organisation en packages permettent de structurer efficacement le projet.

Cette architecture rend l'application plus maintenable, facilite l'ajout de nouvelles fonctionnalités et améliore la compréhension globale du code.



4. Fonctionnalités

L'application propose plusieurs fonctionnalités permettant de manipuler des images.
Ces fonctionnalités sont implémentées dans la classe MainController, qui gère les interactions entre l'utilisateur et le modèle.

4.1.Chargement d'image

Le chargement d'image est réalisé à l'aide de la classe FileChooser.
Lorsque l'utilisateur clique sur le bouton d'ouverture, la méthode openImage() du contrôleur est appelée.
Cette méthode permet de sélectionner un fichier image, puis de le charger sous forme d'objet Image de JavaFX.
L'image est ensuite stockée dans la classe ImageModel via la méthode setImage() et affichée dans le composant ImageView.
En parallèle, un objet TaggableImage est créé afin d'associer des informations supplémentaires à l'image (tags, filtres, etc.).

4.2.Application de filtres

Les filtres sont appliqués via des méthodes du contrôleur telles que applyGray(), applySepia(), applyPrewitt() et applySwap().
Chaque méthode appelle la fonction :
model.applyFilter(new NomDuFiltre())
Le modèle ImageModel applique alors le filtre en utilisant la méthode apply() définie dans l'interface

Filter.
Chaque filtre parcourt l'image pixel par pixel grâce à PixelReader et PixelWriter, et génère une nouvelle image modifiée.
L'image résultante est ensuite affichée dans l'interface.

4.3.Transformation d'image

Les transformations (rotation et symétrie) sont réalisées à l'aide de la classe ImageTransform.
Le contrôleur appelle la méthode :
ImageTransform.apply(image, direction) où direction peut être LEFT, RIGHT, HORIZONTAL ou VERTICAL.
Le résultat de la transformation est ensuite mis à jour dans ImageModel et affiché dans ImageView.

4.4.Gestion des tags

Les tags sont gérés via la classe TaggableImage.
Lorsque l'utilisateur saisit un texte dans le champ prévu et clique sur le bouton, la méthode addTag() est appelée.
Le tag est récupéré depuis le champ TextField, puis ajouté à une liste interne grâce à la méthode addTag().
Les tags sont ensuite stockés dans l'objet associé à l'image.

4.5.Sauvegarde des données

La sauvegarde est réalisée grâce à la classe JsonUtil.
La méthode saveData() du contrôleur appelle : JsonUtil.save(taggableImage, "metadata.json")
Cette méthode utilise la bibliothèque Jackson pour convertir l'objet TaggableImage en format JSON et l'écrire dans un fichier.
Cela permet de persister les informations liées à l'image.

4.6.Sécurité (chiffrement et déchiffrement)

Le chiffrement est réalisé via la classe CryptoUtil.
Lorsque l'utilisateur clique sur le bouton "Encrypt", la méthode encryptImage() est appelée.
Un mot de passe est demandé via un TextInputDialog, puis la méthode suivante est utilisée :
CryptoUtil.encrypt(image, password)
Cette méthode transforme le mot de passe en hash (SHA-256), puis utilise SecureRandom pour mélanger les pixels de l'image.
Le déchiffrement fonctionne de manière similaire avec la méthode decrypt(), en utilisant le même mot de passe pour reconstruire l'image.
Les opérations de chiffrement et déchiffrement sont également enregistrées dans la liste des filtres appliqués.

4.7.Interface utilisateur

L'interface graphique est conçue à l'aide de JavaFX et du fichier FXML.
Elle se compose de plusieurs éléments :

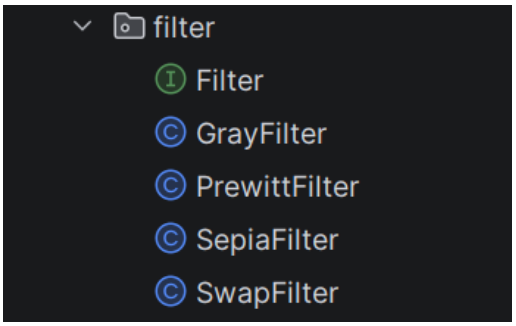
- une barre supérieure contenant les boutons (chargement, filtres, transformations, chiffrement)
- un champ de texte permettant d'ajouter des tags

- une zone centrale (ImageView) permettant d'afficher l'image

Le style visuel est défini à l'aide d'un fichier CSS, permettant d'améliorer l'apparence avec des couleurs, des effets et des animations (hover sur les boutons).
Cette interface offre une utilisation simple et intuitive.

5. Détails techniques – Filtres d'image

Les filtres d'image sont implémentés à l'aide d'une interface commune appelée Filter.
Chaque filtre redéfinit la méthode apply(Image image), qui prend une image en entrée et retourne une nouvelle image transformée.
Le traitement est réalisé pixel par pixel grâce aux classes PixelReader et PixelWriter de JavaFX.
PixelReader permet de lire la couleur d'un pixel, tandis que PixelWriter permet d'écrire la nouvelle couleur dans l'image résultante.



5.1.Filtre Gray (niveaux de gris)

Le filtre Gray permet de convertir une image en niveaux de gris.
Pour chaque pixel, les composantes Rouge (R), Vert (G) et Bleu (B) sont récupérées. Une moyenne est ensuite calculée : Moyenne=(R+G+B)/3.
Cette moyenne est utilisée pour définir les nouvelles composantes du pixel :
R = G = B = moyenne
Cela produit une couleur grise, car lorsque les trois composantes sont égales, le pixel est perçu comme un gris. L'opacité du pixel original est conservée.
Ce filtre permet de simplifier l'image en supprimant les informations de couleur.

5.2.Filtre Prewitt (détection de contours)

Le filtre Prewitt permet de détecter les contours d'une image.
Il repose sur le calcul des variations de luminosité entre pixelsvoins. Deux gradients sont calculés :
gx : variation horizontale
gy : variatioon verticale
Ces gradients sont combinés pour obtenir l'intensité du contour :
$g = \sqrt{(gx^2 + gy^2)}$
Plus la valeur de g est élevée, plus le contour est visible.

La méthode getBrightness() est utilisée pour travailler directement sur la luminosité du pixel. Le résultat est une image en niveaux de gris mettant en évidence les bords.

5.3.Filtre Sepia

Le filtre Sepia donne à l'image un aspect ancien, similaire aux photographies vintage. Pour chaque pixel, de nouvelles valeurs RGB sont calculées à partir des anciennes :

R’ = 0.393R + 0.769G + 0.189B
G’= 0.349R + 0.686G + 0.168B
B’=0.272R + 0.534G + 0.131B

La fonction Math.min est utilisée pour éviter que les valeurs dépassent 1.0. L’opacité de couleurs pour produire une teinte chaude.

5.4.Filtre Swap

Le filtre Swap modifie les couleurs en échangeant les composantes RGB. Pour chaque pixel :

- Rouge devient Vert
- Vert devient Bleu
- Bleu devient Rouge

Cette permutation permet de créer un effet visuel original sans modifier la structure de l’image.

5.5.Transformations d’image

Les transformations d’image sont réalisées à l’aide de la classe ImageTransform.
Cette classe permet de modifier la position des pixels sans changer leurs couleurs.
Plusieurs transformations sont disponibles :
Rotation gauche (LEFT) : l’image est tournée de 90° vers la gauche.
Chaque pixel change de position selon une transformation des coordonnées :
x devient y
y devient largeur – x
Rotation droite (RIGHT) : l’image est tournée de 90° vers la droite.
Symétrie horizontale(HORIZONTAL) : l’image est inversée de gauche à droite. Les pixels sont déplacés horizontalement.
Symétrie verticale (VERTICAL) : l’image est inversée de haut en bas. Ces transformations sont appliquées en parcourant l’image pixel par pixel, puis en réaffectant chaque pixel à une nouvelle position.

5.6.Sérialisation des données (JSON)

La sauvegarde des données est réalisée grâce à la classe JsonUtil, qui utilise la bibliothèque Jackson.
Cette classe permet de convertir un objet Java (TaggableImage) en fichier JSON.
Le fichier contient :

- le chemin de l’image
- la liste des tags
- la liste des filtres appliqués

5.7.Chiffrement et déchiffrement

Le chiffrement et le déchiffrement sont implémentés dans la classe CryptoUtil.

Le mot de passe saisi par l’utilisateur est transformé en hash à l’aide de l’algorithme SHA-256.
Ce hash est utilisé comme graine pour initialiser un générateur pseudo-aléatoire SecureRandom.

Lors du chiffrement, chaque pixel de l’image est remplacé par un pixel choisi aléatoirement dans l’image.

Lors du déchiffrement, le même mot de passe permet de générer la même suite de positions aléatoires, ce qui permet de reconstruire l’image.

Cette méthode reste simple et ne garantit pas une sécurité forte.

```
public class CryptoUtil { // classe utilitaire permettant de chiffrer et déchiffrer une image
    public static Image encrypt(Image image, String password) { // methode de chiffrement
        int width = (int) image.getWidth(); // recuperation des dim de limage
        int height = (int) image.getHeight();
        WritableImage newImage = new WritableImage(width, height); // creation dune nvlle image pour stocker le res
        PixelReader reader = image.getPixelReader(); // permet de lire les px de limage originale
        PixelWriter writer = newImage.getPixelWriter(); // permet decrire dans la nvlle image
        try {
            // transformer password en hash
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(password.getBytes());
            // generateur pseudo-aleatoire base sur le hash
            SecureRandom random = SecureRandom.getInstance("SHA1PRNG");
            random.setSeed(hash);
            // melanger les pixels
            for (int y = 0; y < height; y++) {
                for (int x = 0; x < width; x++) {
                    // generation des cordonnes aleatoires
                    int rx = random.nextInt(width);
                    int ry = random.nextInt(height);
                    // recuperation dun px aleatoire
                    Color color = reader.getColor(rx, ry);
                    // ecriture dans la nvlle pos
```

```
// methode de dechiffrement
    public static Image decrypt(Image image, String password) {
        int width = (int) image.getWidth();
        int height = (int) image.getHeight();
        WritableImage newImage = new WritableImage(width, height);
        PixelWriter writer = newImage.getPixelWriter();
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(password.getBytes());

            SecureRandom random = SecureRandom.getInstance("SHA1PRNG");
            random.setSeed(hash);
            for (int y = 0; y < height; y++) {
                for (int x = 0; x < width; x++) {

                    int rx = random.nextInt(width);
                    int ry = random.nextInt(height);

                    Color color = image.getPixelReader().getColor(x, y);
```

5.8.Gestion des erreurs

L'application intègre une gestion basique des erreurs afin d'éviter certains dysfonctionnements. Par exemple :

- vérification qu'une image est chargée avant d'appliquer un filtre ou une transformation
- vérification que le champ de saisie des tags n'est pas vide
- gestion des exceptions lors de la sauvegarde des données JSON

Cependant, cette gestion reste limitée et pourrait être améliorée avec des messages d'erreur plus explicites pour l'utilisateur.

6. Limites du projet

Malgré ses fonctionnalités, l'application présente certaines limites :

- Le chiffrement n'est pas totalement sécurisé (simple mélange de pixels)
- Il n'y a pas de système de restauration de l'image originale
- Les filtres ne peuvent pas être cumulés (toujours appliqués sur l'image originale)
- L'interface reste simple et peut être améliorée
- Aucun système de gestion d'erreurs avancé

Ces limites pourraient être améliorées dans une version future.

7. Conclusion

Ce projet a permis de mettre en pratique plusieurs concepts de la programmation orientée objet, notamment le modèle MVC.
L'utilisation de JavaFX a facilité la création d'une interface graphique interactive et intuitive.
Les fonctionnalités développées, telles que les filtres, les transformations, la gestion des tags et le chiffrement, offrent une application complète de manipulation d'images.

Ce projet peut être amélioré en ajoutant de nouvelles fonctionnalités, comme :

- davantage de filtres
- une meilleure sécurité
- une interface plus avancée

Il constitue une base solide pour le développement d'applications graphiques en Java.

8. Bibliographie

- Université de Limoges. (2024). Cours JavaFX – Interfaces graphiques : https://community-sciences.unilim.fr/pluginfile.php/253533/mod_resource/content/0/06-JavaFX.pdf
- OpenJFX. (2024). Documentation officielle JavaFX : https://openjfx.io/
- Oracle. (2024). Documentation Java SE : https://docs.oracle.com/en/java/
- FasterXML. (2024). Jackson JSON Processor : https://github.com/FasterXML/jackson
- Programming Knowledge. (2024)JavaFX Tutorial for Beginners : https://www.youtube.com/watch?v=9XJicRt_FaI
- Google : https://www.google.com/